BrainAI

A spiking, multimodal cognitive architecture with continual learning. Built by Mohammed Zahid.

BrainAI is one model that perceives text, images, video, audio and touch; routes each input through a reflex arc, a subconscious pathway and a gated conscious pathway; keeps five kinds of memory; updates its own weights while running; and consolidates what it learned during an offline sleep cycle. It emits both language and actuator commands, so the same core drives speech and motion.

The default deployment name is `BrainAI`. Any instance can be renamed — `model.rename("Javri")` — and authorship travels with the weights regardless.

```
pip install torch
python train.py --steps 400          # pretrain (CPU, ~10 min)
python continual.py --steps 120      # the anti-forgetting measurement
python demo.py                       # every capability, end to end
python -m pytest tests -q            # 45 tests
```

What is actually novel here

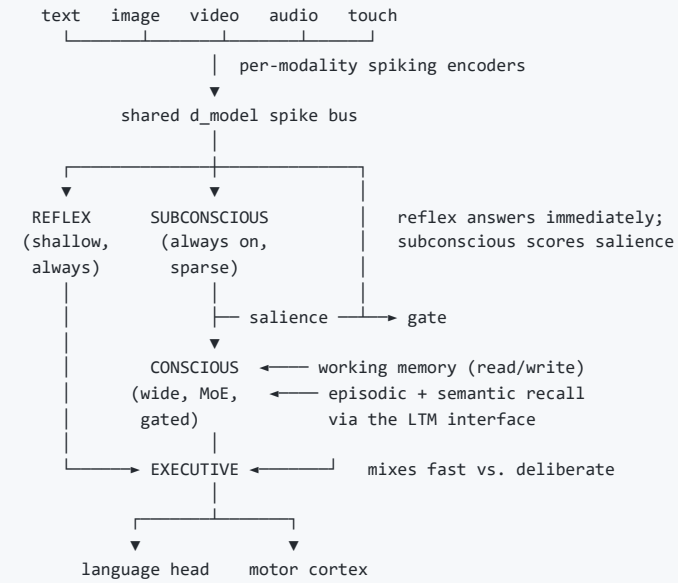
Spiking networks, transformers, retrieval memory, EWC, MoE and sleep replay all exist separately. What is assembled here and what is not:

Implemented and measured

- **AMSA**, an activation whose threshold and gain are driven by each channel's learned importance, not just by learned constants.
- **Latency-tiered pathways** with a gate that genuinely skips computation at inference rather than masking it.
- **A memory split along the differentiability line** — neural state for reasoning, non-differentiable stores for facts — so post-training knowledge cannot corrupt a weight an older skill depends on.
- **Bounded inference-time weight updates** with per-parameter plasticity, a trust region, and automatic rollback.
- **Sleep** that replays raw tokens, merges redundant memories, and re-anchors.

Proposed, not proven. AMSA has no evidence of beating SwiGLU — see the benchmark. The `full` preset is a written-down specification, not a trained model. Nothing here has been run on a robot.

Architecture



The three pathways

Pathway	Runs	Cost	Job
Reflex	always	1 shallow stack	Have *something* correct-enough available now. Never consults memory.
Subconscious	always	sparse	Filter, predict, and decide whether thinking is worth it.
Conscious	when salience clears threshold	wide, expert-routed	Reasoning, planning, memory integration.

The gate is **soft during training** so the salience head learns *when* deliberation pays, and **hard at inference** so skipping is a real saving — `run_gated` selects the batch rows that need it and runs only those:

```
threshold 0.0  -> conscious ran on 100% of the batch
threshold 0.5  -> conscious ran on  50% of the batch
threshold 0.99 -> conscious ran on   0% of the batch
```

The five memories

Memory	Backing	Differentiable	Holds
Sensory	deque	no	Last few frames, for temporal integration
Short-term	spiking LSTM	**yes**	The recent event stream, pooled
Working	attention over slots	**yes**	The reasoning scratchpad
Episodic	`VectorStore` buffer	no	Experiences, importance-ranked
Semantic	`VectorStore` + graph	no	Concepts and the edges between them

The differentiability split is the anti-forgetting strategy. Facts learned after training go into the *stores*, where writing them cannot damage a weight some older skill depends on. Only slow, importance-gated consolidation ever moves knowledge from a store into the weights.

Because the stores are registered buffers, they are saved and restored with the checkpoint — a robot's episodic history survives a restart.

Two design decisions in there are load-bearing and easy to get wrong:

- **The key/value storage projections are frozen random projections, not learned.** Every key already in the store was written with those weights. If they were trained, each update would silently invalidate the entire store, because old keys and new queries would stop living in the same space. A random projection preserves the cosine geometry retrieval needs.
- **Retrieval is non-differentiable on purpose.** A bad memory can mislead the model but can never corrupt a gradient. The cost is that the importance scorer gets no signal from the language loss, so it is trained during sleep against a prioritised-replay target instead: an episode is important to the degree the model still gets it wrong.

Continual learning: the measurement

The claim is "learns new things without forgetting old ones." That is testable, so it is tested. `continual.py` pretrains on English, then trains on French three ways from identical weights, and measures what happened to English.

English and French at byte level have clearly different statistics, so forgetting has an unambiguous number attached.

Every arm sees the same number of French sequences over the same number of optimizer steps. The `full` arm additionally replays English from its buffer, which costs ~1.5× compute per step plus the memory to hold the buffer. That cost is the price of the method and is reported, not hidden.

AMSA — Adaptive Memory Spike Activation

```
f(x) = z · σ(α·z) · (1 − e^(−β|z|)) · (1 + γ·m),    z = x − θ

θ = θ₀ + λ·p        p = per-channel plasticity
α = softplus(α₀ + δ·c)   c = per-channel confidence
β = softplus(β₀)
```

Term	Effect
`z = x − θ`	Adaptive threshold — the neuron responds once its own learned threshold is exceeded.
`σ(α·z)`	Smooth gate; keeps gradients well-conditioned where a hard cutoff would not.
`1 − e^(−β`	z
`1 + γ·m`	Memory modulation — channels carrying important information amplify.

`θ₀, α₀, β₀, γ, λ, δ` are learned per channel. `m`, `p`, `c` are **state**, written by the plasticity controller, never by gradient descent — which is what makes the activation's *shape* follow the network's own estimate of which channels are load-bearing. Confident channels sharpen; plastic channels stay easy to move.

`1 − e^(−β|z|)` is computed as `-expm1(-β|z|)` to stay accurate for small `β|z|`.

Does AMSA actually help?

Spiking core

Spikes are real Heaviside steps — the forward pass is genuinely binary, so the reported spike rate is a true sparsity number and not a soft approximation. Training uses an arctan surrogate on the backward pass only.

Neurons are **adaptive** LIF: each spike raises that neuron's own firing threshold, which decays over `tau_adapt`. A neuron that just fired is harder to re-fire, giving the layer temporal memory across timesteps with no recurrent weights.

Two implementation facts worth knowing before changing anything:

- **`timesteps >= 3` in every preset.** Adaptation is `a[t] = ρ·a[t−1] + s[t−1]` with `a[0] = s[0] = 0`, so at `T = 2` the decay only ever multiplies zeros and threshold adaptation is inert. `T = 3` is the smallest useful SNN. (Tested: `test_alif_needs_three_timesteps_for_adaptation`.)
- **The residual stream stays analogue.** Only the sublayer interiors are binary. Making the residual stream itself spiking destroys the gradient path and the model does not train.

Attention offers `softmax` (default) or `sparsemax`, which produces exactly-zero attention weights and keeps the attention pattern sparse too.

Scale presets

`BrainConfig.for_scale(...)` — the architecture is described once and instantiated at four sizes.

Scale	Layers	d_model	T	Params (est.)	Runs on
`nano`	17	128	3	~5M actual	a laptop CPU
`small`	33	256	3	~0.03B	a laptop CPU, slowly
`base`	85	768	4	~0.6B	one GPU
`full`	382	4096	8	**~1.7T**	not this machine

`full` is the humanoid specification: 96 conscious layers at width 65,536, 64 subconscious layers at 32,768, and so on. It is in the repo because the design should be written down honestly — not because it can be trained here. At that size the memory stores also need an external ANN index rather than a dense tensor; `VectorStore` keeps the same API either way.

```
python -c "from brainai import BrainConfig; print(BrainConfig.for_scale('full').describe())"
```

Learning while running

```
model.eval()
model.learn_online(tokens, reward=1.0)   # weights change, in eval mode
model.sleep(cycles=8)                    # consolidate offline
```

`learn_online` is deliberately conservative, because unbounded online updates destabilise a model within a few hundred steps:

1. Gradients are scaled per-parameter by **plasticity** = `1 - normalised Fisher`. Weights that mattered to past tasks barely move.
2. The applied update is clipped to a **trust region** — a fraction of the parameter norm, measured after plasticity scaling.
3. The loss is **re-evaluated**; if the step made things worse it is rolled back.

The Fisher is normalised by a **high quantile, not the max**. The Fisher is heavy-tailed, so `f / f.max()` is near zero for almost every weight and nothing ends up protected — a bug that looks like a working implementation until you print the protected fraction.

`sleep()` then replays stored token sequences against current weights with the EWC penalty active, merges near-duplicate memories, lets unimportant ones decay, and re-anchors. Replay stores **raw tokens, not embeddings** — replaying an embedding only rehearses the layers above where it was captured.

Distilling from Qwen

`distill.py`. One constraint determines the design: **you cannot KL-match two models with different tokenizers.** BrainAI's byte tokenizer has 260 symbols; Qwen's BPE has ~150k. Their output distributions live in different spaces.

```
python distill.py --mode sequence --dry-run          # validate the pipeline offline
python distill.py --teacher Qwen/Qwen2.5-0.5B --mode sequence
python distill.py --teacher Qwen/Qwen2.5-0.5B --mode logits --share-tokenizer
```

- **`sequence`** (default) — the teacher generates text, the student trains on it. Tokenizer-agnostic, works with any teacher. What it cannot transfer is the teacher's uncertainty: the student sees the sampled token, not the distribution it came from.
- **`logits`** — rebuilds the student on the teacher's vocabulary, then matches full distributions with temperature-scaled KL. Transfers much more, and costs you the byte tokenizer, which for a robot also handling tactile and telemetry byte streams is a real loss.

Distillation gives the student language knowledge. Perception, planning and motor control need their own training; language distillation alone will not produce a robot.

Naming and attribution

```
model.rename("Javri")     # or anything else
model.identity.author     # always "Mohammed Zahid" - a read-only property
```

Authorship is stamped into every module, every config, and every checkpoint. `BrainAI.load` verifies it and **refuses to load weights whose attribution has been stripped or altered**, while renaming passes cleanly.

Repository layout

```
brainai/
  identity.py       attribution, renaming, tamper checking
  config.py         scale presets, section specs
  activation.py     AMSA, GatedAMSA
  neurons.py        ALIF, surrogate gradients, spiking LSTM
  attention.py      spike attention, sparsemax, RMSNorm
  blocks.py         spiking blocks, MoE routing, expert growth
  encoders.py       text / image / video / audio / touch
  memory.py         the five memories + the hippocampal controller
  pathways.py       reflex, subconscious, conscious, executive, motor
  plasticity.py     Fisher, EWC, online updates, growth
  consolidation.py  replay buffer, sleep
  tokenizer.py      byte-level
  data.py           corpora, evaluation
  model.py          the assembled model
train.py            pretraining
continual.py        the anti-forgetting benchmark
distill.py          Qwen distillation
demo.py             every capability, end to end
benchmarks/         AMSA vs GELU vs SwiGLU
tests/              45 tests
```

Checkpoints are ~3× the model size: the plasticity controller stores a Fisher estimate, an anchor and a plasticity value per weight. That is the cost of the continual-learning machinery, and it is why a 5M-parameter `nano` checkpoint is ~89 MB.

Limitations

- `nano` is 5M parameters trained on one book for 400 steps. It is a working demonstration of the architecture, not a capable language model.
- AMSA is an original proposal with no evidence of superiority over established activations. Treat the benchmark as a sanity check, not a result.
- The energy argument for spiking is a hardware argument. On a CPU or GPU, binary activations do not save wall-clock time — dense matmuls run anyway. The sparsity numbers are what neuromorphic hardware would exploit; here they are a proxy.
- `full` has never been instantiated.
- Nothing has been run on a robot. `MotorCortex` emits bounded joint targets; closing the loop needs a simulator, a safety layer, and real robotics data.
- Continual learning is measured on a two-task language pair. Two tasks is the smallest honest test, not a thorough one.

Built by **Mohammed Zahid**.